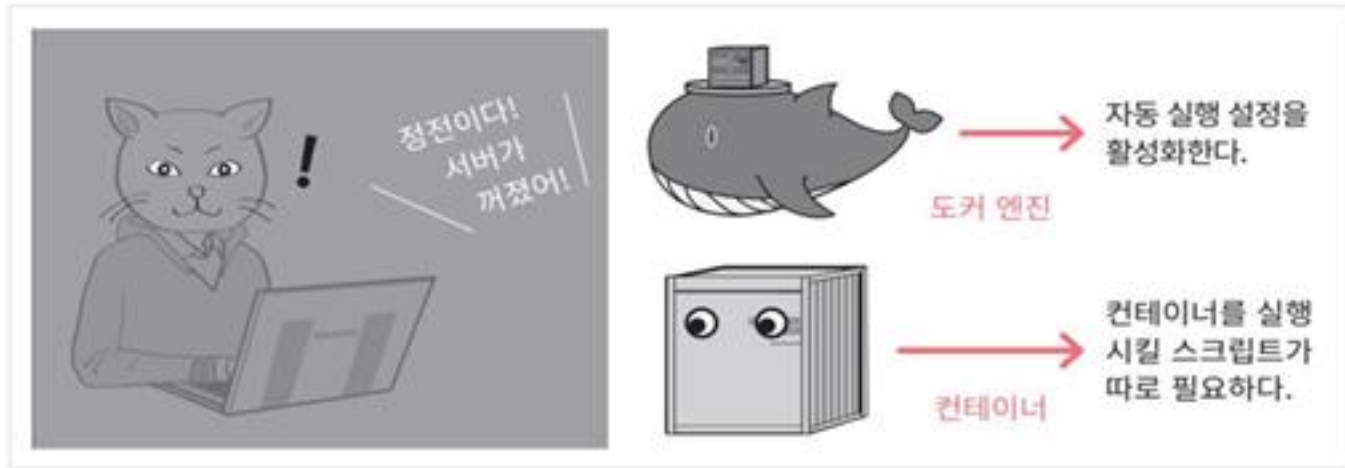


컨테이너 실행

1절 - 도커 엔진 시작하기와 종료하기

도커 엔진 시작/종료

- 도커 엔진 자동 실행
 - 컨테이너를 실행하지 않으면 컴퓨터의 리소스는 거의 차지하지 않음
- 도커 엔진이 종료되면 컨테이너는 모두 정지됨
- 도커엔진은 자동으로 구동되나 컨테이너는 자동으로 구동되지 않음



도커 엔진 시작/종료

OS	도커 엔진 시작	도커 엔진 종료	자동실행설정
윈도우	시작 → Docker Desktop 선택	도커 데스크톱 창 닫기	기본으로 자동 구동
macOS	파인더 → '애플리케이션' 폴더 → Docker Desktop 선택	상태 막대에서 도커 클릭 → Quit Docker Desktop 선택	상태막대에서 도커 클릭 → Preference → Start Docker Desktop when you login 선택
리눅스	<code>sudo systemctl start docker</code>	<code>sudo systemctl stop docker</code>	<code>sudo systemctl enable docker</code>

컨테이너 실행

2절 - 컨테이너의 기본적인 사용 방법

도커 기본 명령어

- 도커 명령의 구성

docker	상위명령	하위명령	[옵션]	대상	[인자]
	container	run	명령의 세부 설정 시 이용		
	image	pull	-d : 백그라운드 실행		
	.	start	-i or -t : 대화형 실행		
	.	.		명령을 수행할 구체적인 대상	
	.	.			
		start/stop/run			대상에 전달할 값 지정
		명령인 경우			
		container 생략			
		가능			

도커 기본 명령어

- <실습> 버전 확인

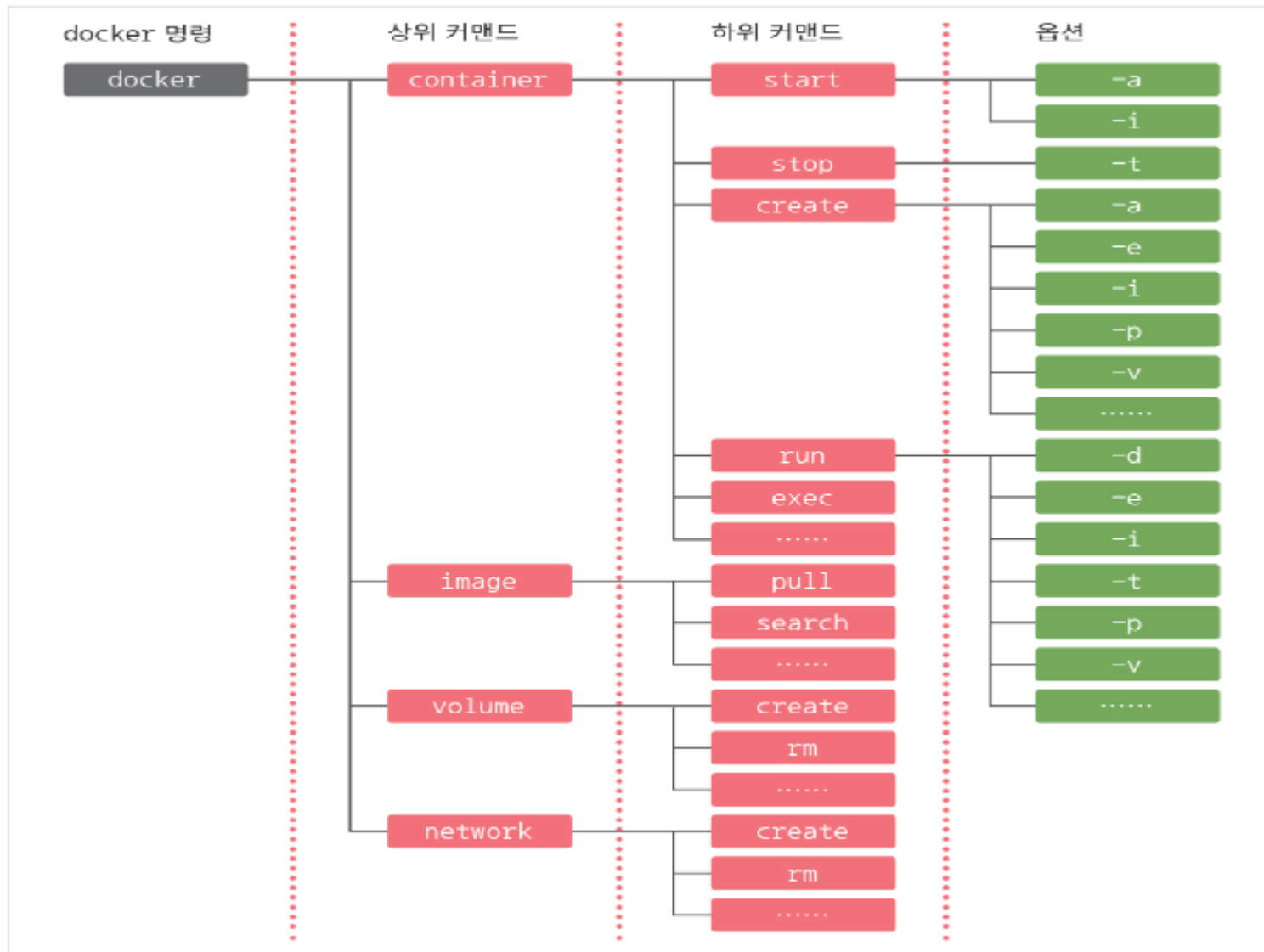
- docker version

```
PS C:\Users\khali> docker version
Client:
 Cloud integration: v1.0.31
 Version: 20.10.23
 API version: 1.41
 Go version: go1.18.10
 Git commit: 7155243
 Built: Thu Jan 19 17:43:10 2023
 OS/Arch: windows/amd64
 Context: default
 Experimental: true

Server: Docker Desktop 4.17.1 (101757)
Engine:
 Version: 20.10.23
 API version: 1.41 (minimum version 1.12)
 Go version: go1.18.10
 Git commit: 6051f14
 Built: Thu Jan 19 17:32:04 2023
 OS/Arch: linux/amd64
 Experimental: false
containerd:
 Version: 1.6.18
 GitCommit: 2456e983eb9e37e47538f59ea18f2043c9a73640
runc:
 Version: 1.1.4
 GitCommit: v1.1.4-0-g5fd4c4d
docker-init:
 Version: 0.19.0
 GitCommit: de40ad0
```

대표적인 docker 명령

- 컨테이너를 다루는 도커 명령



대표적인 docker 명령

- 컨테이너 제어 명령

docker [container] 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
start	컨테이너를 실행	O	-i
stop	컨테이너를 정지	O	거의 사용하지 않음
create	도커 이미지로부터 컨테이너를 생성	O	--name -e -p -v
run	도커 이미지를 내려받고 컨테이너를 생성해 실행함(다운 로드는 필요한 경우에만). docker image pull, docker container create, docker container start라는 세 개의 명령을 하나로 합친 것과 같다.	O	--name -e -p -v -d -i -t
rm	정지 상태의 컨테이너를 삭제	O	-f -v
exec	실행 중인 컨테이너 속에서 프로그램을 실행	O	-i -t
ls	컨테이너 목록을 출력	*1	-a
cp	도커 컨테이너와 도커 호스트 간에 파일을 복사	O	거의 사용하지 않음
commit	도커 컨테이너를 이미지로 변환	O	거의 사용하지 않음

*1: 생략형은 docker ps

대표적인 docker 명령

- 이미지 제어 명령

docker image 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
pull	도커 허브 등의 리포지토리에서 이미지를 내려받음	O	거의 사용하지 않음
rm	도커 이미지를 삭제	*2	거의 사용하지 않음
ls	내려 받은 이미지의 목록을 출력	X	거의 사용하지 않음
build	도커 이미지를 생성	O	-t

*2: 생략형은 docker rmi

대표적인 docker 명령

- 볼륨 제어 명령

docker volume 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
create	볼륨을 생성	X	--name
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	-a
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

대표적인 docker 명령

- 네트워크 제어 명령

docker network 하위명령어 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
connect	컨테이너를 도커 네트워크에 연결	X	거의 사용하지 않음
disconnect	컨테이너의 도커 네트워크 연결을 해제	X	거의 사용하지 않음
create	도커 네트워크를 생성	X	거의 사용하지 않음
inspect	도커 네트워크의 상세 정보를 출력	X	거의 사용하지 않음
ls	도커 네트워크의 목록을 출력	X	거의 사용하지 않음
prune	현재 컨테이너가 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

대표적인 docker 명령

- 기타 명령어

그 밖의 상위 커맨드

상위 커맨드	내용
checkpoint	현재 상태를 일시적으로 저장한 후, 나중에 해당 시점의 상태로 되돌릴 수 있다. 현재 실험적 기능이다.
node	도커 스웜의 노드를 관리하는 기능
plugin	플러그인을 관리하는 기능
secret	도커 스웜의 비밀값 정보를 관리하는 기능
service	도커 스웜의 서비스를 관리하는 기능
stack	도커 스웜 또는 쿠버네티스에서 여러 개의 서비스를 합쳐 구성한 스택을 관리하는 기능
swarm	도커 스웜을 관리하는 기능
system	도커 엔진의 정보를 확인하는 기능

대표적인 docker 명령

- 단독 명령어

단독 커맨드⁹

단독 커맨드	내용	주요 옵션
login	도커 레지스트리에 로그인	-u -p
logout	도커 레지스트리에 로그아웃	거의 사용하지 않음
search	도커 레지스트리를 검색	거의 사용하지 않음
version	도커 엔진 및 명령행 도구의 버전을 출력	거의 사용하지 않음

컨테이너 실행

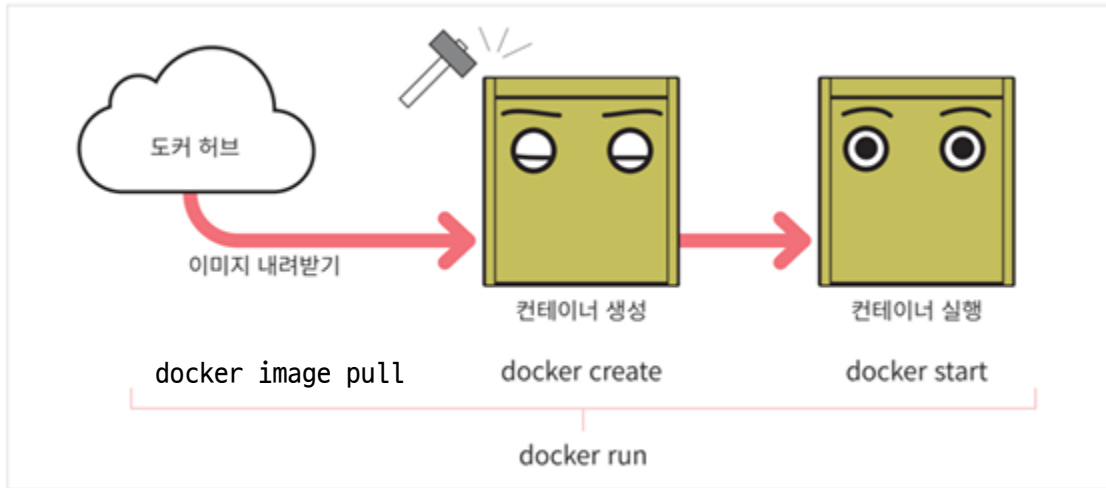
3절 - 컨테이너의 생성과 삭제, 실행, 정지

도커 생성 및 실행

● 도커 실행

docker [container] run 옵션

- 도커 생성 및 실행 기능, 필요시 이미지 다운로드 기능도 수행
- docker image pull + docker create + docker start

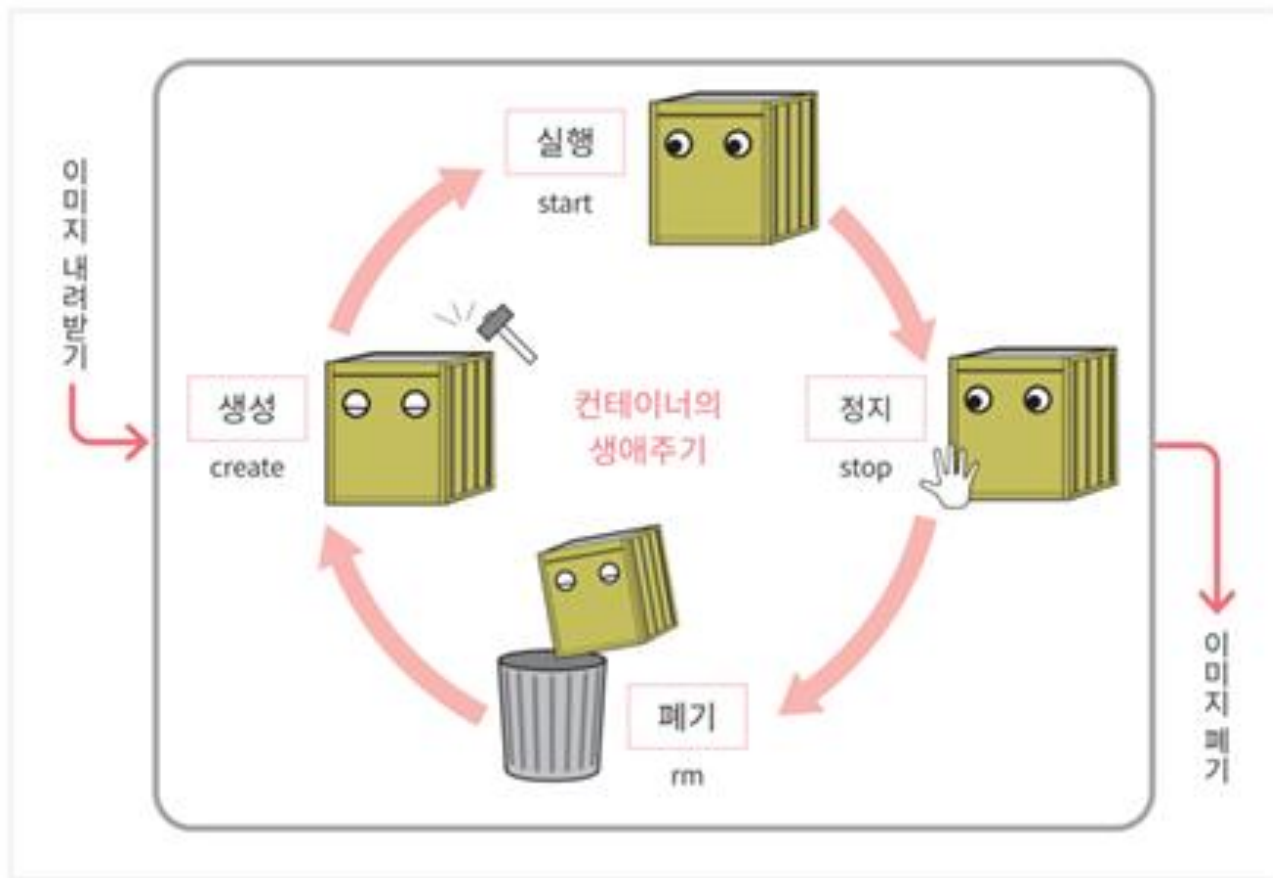


- docker run 커맨드는 세가지 역할을 수행함

도커 생성 및 실행

● 도커 컨테이너의 생애주기

- 동작 중인 컨테이너는 삭제 불가능
- 컨테이너 정지 후 삭제
 - `docker stop & docker rm`



도커 생성 및 실행

- 컨테이너 생성 및 실행

```
docker run 옵션 이미지 [인자]
```

주요 옵션

옵션 형식	내용
--name 컨테이너_이름	컨테이너 이름을 지정함
-p 호스트_포트번호:컨테이너_포트번호	포트 번호를 지정함
-v 호스트_디스크:컨테이너_디렉터리	볼륨을 마운트함
--net=네트워크_이름	컨테이너를 네트워크에 연결함
-e 환경변수_이름=값	환경변수를 설정함
-d	백그라운드로 실행함
-i	컨테이너와 대화형 모드로 연결함
-t	컨테이너에 터미널 연결함
-help	사용 방법 안내 메시지를 출력함

-p는 --publish, -v는 --volume, -e는 --env, -d는 --detach, -i는 --interactive, -t는 --tty의 생략형이다.

도커 생성 및 실행

- 컨테이너 정지 명령

```
docker stop 컨테이너명
```

- 컨테이너 삭제 명령

```
docker rm 컨테이너명
```

도커 생성 및 실행

- 컨테이너 목록 확인 명령

```
docker ps [옵션]  
or  
docker container ls [옵션]
```

컨테이너 목록의 주요 항목

항목	내용
CONTAINER ID	컨테이너 식별자. 무작위 문자열이 할당된다. 본래는 64글자이지만 앞에서부터 12글자만 출력한다. 이 12글자만으로도 식별자 역할을 수행할 수 있다.
IMAGE	컨테이너를 만들 때 사용한 이미지의 이름
COMMAND	컨테이너 실행 시에 실행하도록 설정된 프로그램의 이름. 크게 신경 쓰지 않아도 된다.
CREATED	컨테이너 생성 후 경과된 시간
STATUS	컨테이너의 현재 상태. 실행 중이라면 'Up', 종료된 상태라면 'Exited'가 출력된다.
PORTS	컨테이너에 할당된 포트 번호. '호스트 포트 번호 -> 컨테이너 포트 번호' 형식으로 출력된다. 포트 번호가 동일할 경우 ->의 뒷부분은 출력되지 않는다.
NAMES	컨테이너 이름

도커 생성 및 실행

- <실습> 컨테이너 생성, 실행, 상태 확인 및 종료



- 컨테이너 정보

항목	값
컨테이너 이름	apa000ex1
이미지 이름	httpd

옵션 항목

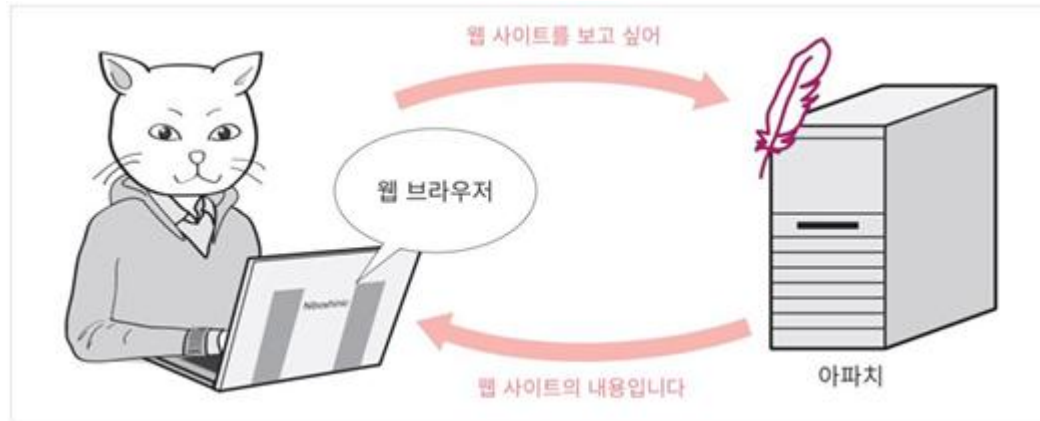
항목	내용
--name apa000ex1	apa000ex1이라는 이름으로 컨테이너를 생성
-d	백그라운드로 실행
httpd	아파치의 이미지 이름. 버전을 지정하지 않았으므로 가장 최신 버전(latest)이 사용된다.

컨테이너 실행

4절 - 컨테이너의 통신

아파치 웹서버

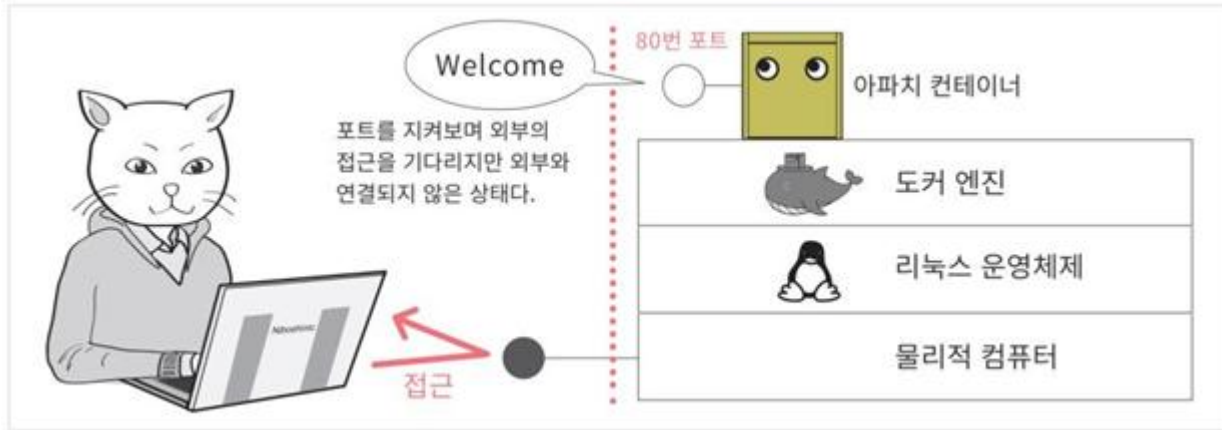
- 아파치 : 웹서버를 제공하는 소프트웨어
- 컨테이너는 기본적으로 외부에서 접근 불가능한 상태로 실행
- 따라서 컨테이너 실행 시 설정 필요
 - docker run 명령의 옵션 이용
 - 실행 후 변경은 불가능



- 컨테이너에서 동작하는 아파치의 웹 사이트를 열람

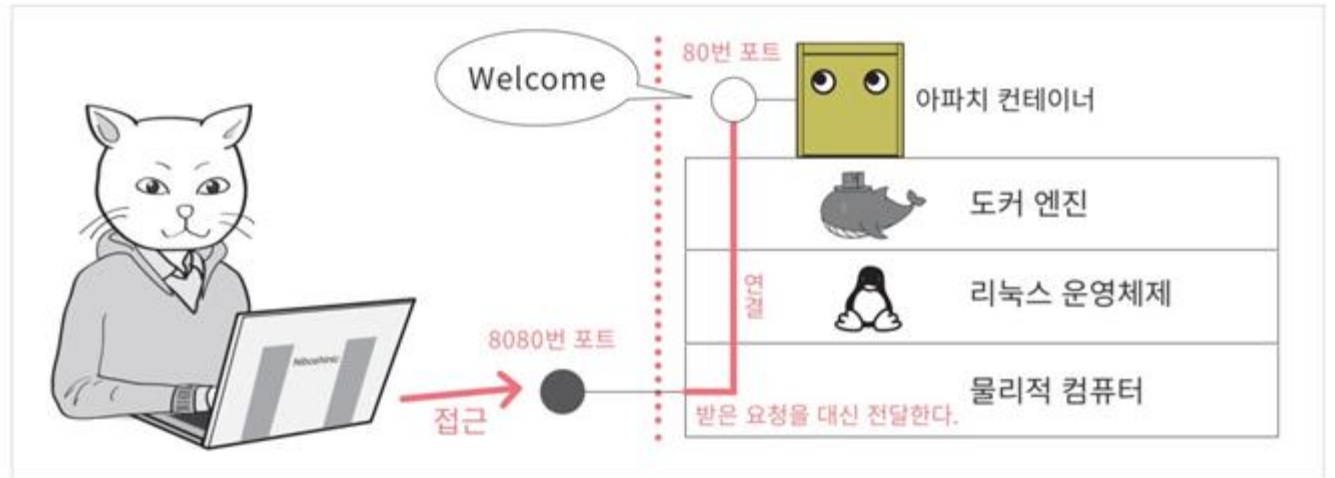
컨테이너와의 통신

- 외부와의 접속을 위해 통신 포트 설정



- 컨테이너의 통신

- 컨테이너를 실행중인 물리적 컴퓨터의 포트를 통해 컨테이너와 통신한다.



컨테이너와의 통신

- 포트 설정

`-p 호스트포트번호:컨테이너_포트번호`

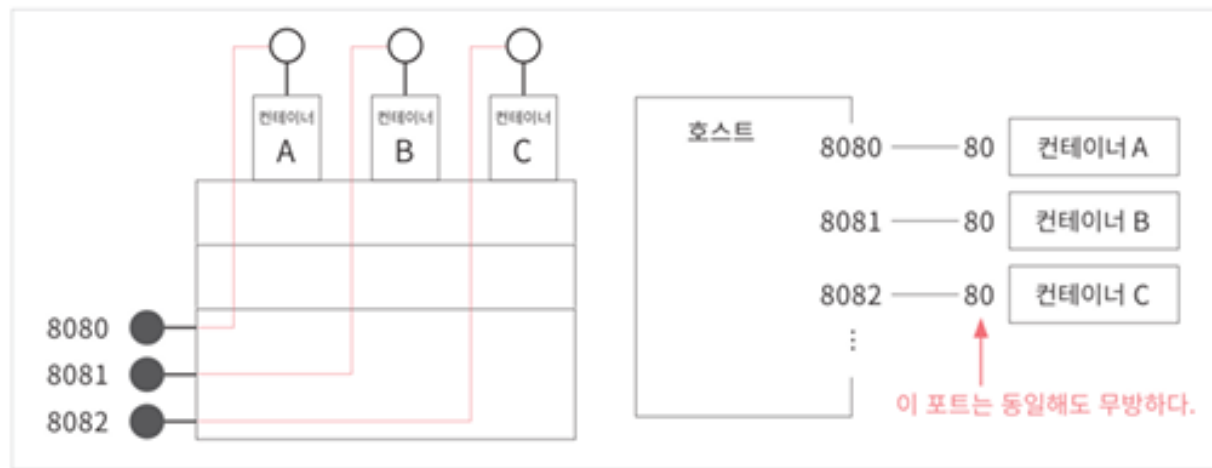
예) `-p 8080:80`

- 여러 개의 웹 서버 컨테이너 구동 시 호스트_포트번호는 다르게 구성

컨테이너 A는 8080

컨테이너 B는 8081

컨테이너 C는 8082

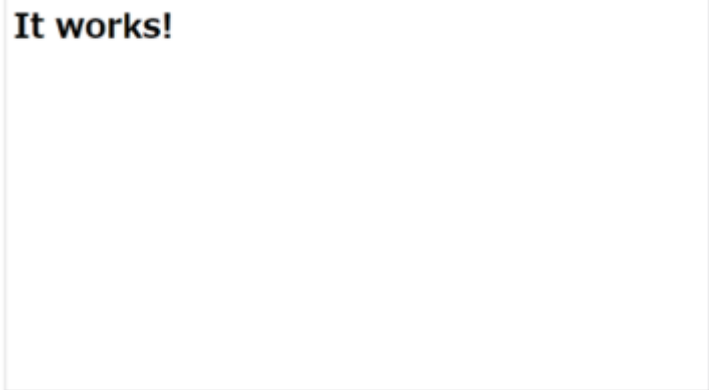


- 컨테이너와 연결하는 호스트의 포트 번호를 겹치지 않게 설정한다.

컨테이너와의 통신

- 웹 브라우저를 이용한 접근

<http://localhost:8080> 로 접근



It works!

- 아파치의 초기 화면

컨테이너와의 통신

- <실습> 통신이 가능한 컨테이너 생성



- 컨테이너 정보

항목	값
컨테이너 이름	apa000ex2
이미지 이름	httpd
포트 설정	8080:80

옵션 항목

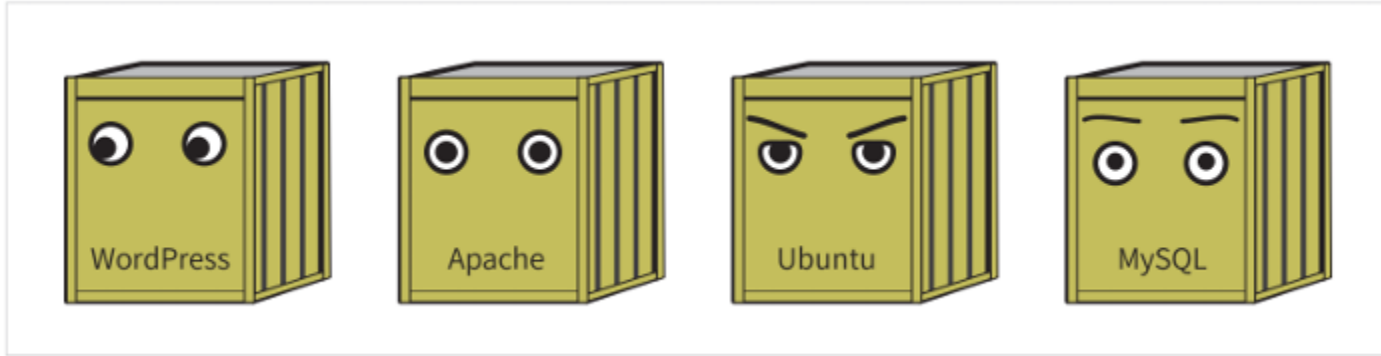
항목	내용
--name apa000ex2	apa000ex2라는 이름으로 컨테이너를 생성
-d	백그라운드로 실행
-p 8080:80	호스트의 포트 8080을 컨테이너 포트 80으로 포워딩
httpd	아파치의 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

컨테이너 실행

5절 - 컨테이너의 생성에 익숙해지기

다양한 유형의 컨테이너

- 아파치 이외에 많이 사용되는 컨테이너



- 다양한 유형의 컨테이너

다양한 유형의 컨테이너

- 리눅스 운영체제 컨테이너

리눅스 운영체제가 담긴 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
ubuntu	우분투	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
centos	CentOS	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
debian	데비안	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
fedora	페도라	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
busybox	BizyBox	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
alpine	알파인 리눅스	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.

다양한 유형의 컨테이너

- 웹서버/데이터베이스 서버용 컨테이너

웹 서버 및 데이터베이스 서버용 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
httpd	Apache	-d로 백그라운드로 실행. -p로 포트 번호 지정
nginx	Nginx	-d로 백그라운드로 실행. -p로 포트 번호 지정
mysql	MySQL	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정
postgres	PostgreSQL	-d를 사용. 실행 시 -e POSTGRES_ROOT_PASSWORD와 같이 루트 패스워드를 지정
mariadb	MariaDB	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정

다양한 유형의 컨테이너

- 프로그램 실행을 위한 런타임 및 그 외 소프트웨어

프로그래밍 언어의 런타임 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
openjdk	자바 런타임	-d를 사용하지 않고, 인자로 java 명령 등을 지정해 도구 형태로 사용한다.
python	파이썬 런타임	-d를 사용하지 않고, 인자로 python 명령 등을 지정해 도구 형태로 사용한다.
php	PHP 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
ruby	루비 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
perl	펄 런타임	-d를 사용하지 않고, 인자로 perl 명령 등을 지정해 도구 형태로 사용한다.
gcc	C/C++ 컴파일러	-d를 사용하지 않고, 인자로 gcc 명령 등을 지정해 도구 형태로 사용한다.
node	Node.js	-d를 사용하지 않고, 인자로 app 명령 등을 지정해 도구 형태로 사용한다.
registry	도커 레지스트리	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다.
wordpress	WordPress	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. MySQL 또는 MariaDB가 필요하다. 접속에 필요한 패스워드는 -e 옵션으로 지정한다.
nextcloud	NextCloud	-d 옵션을 사용해 백그라운드로 실행한다. -p 옵션으로 포트 번호를 지정한다.
redmine	Redmine	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. PostgreSQL 또는 MySQL이 필요하다.

다양한 유형의 컨테이너

- <실습> 아파치 컨테이너 여러 개 실행하기

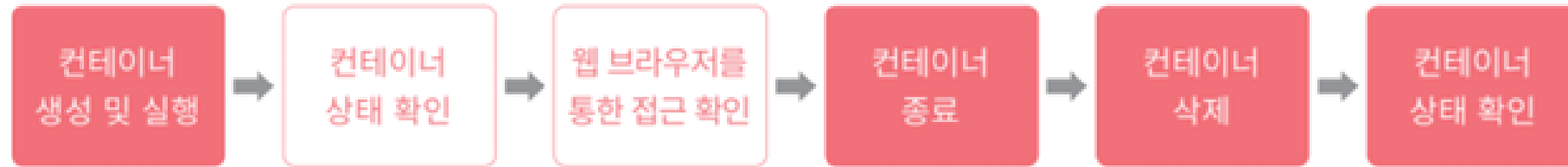


- 컨테이너 정보

항목	값	값	값
컨테이너 이름	apa000ex3	apa000ex4	apa000ex5
이미지 이름	httpd	httpd	httpd
포트 설정	8081:80	8082:80	8083:80

다양한 유형의 컨테이너

- <실습> nginx 컨테이너 실행하기



- 컨테이너 정보

항목	값
컨테이너 이름	nginx000ex6
이미지 이름	nginx
포트 설정	8084:80

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

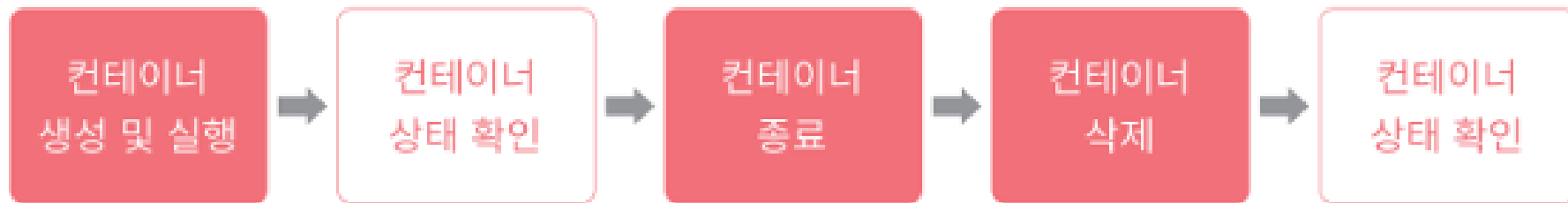
For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

- nginx의 초기 화면

다양한 유형의 컨테이너

- <실습> MySQL 컨테이너 실행하기



- 컨테이너 정보

항목	값
컨테이너 이름	mysql000ex7
이미지 이름	mysql
MySQL 루트 패스워드	myrootpass

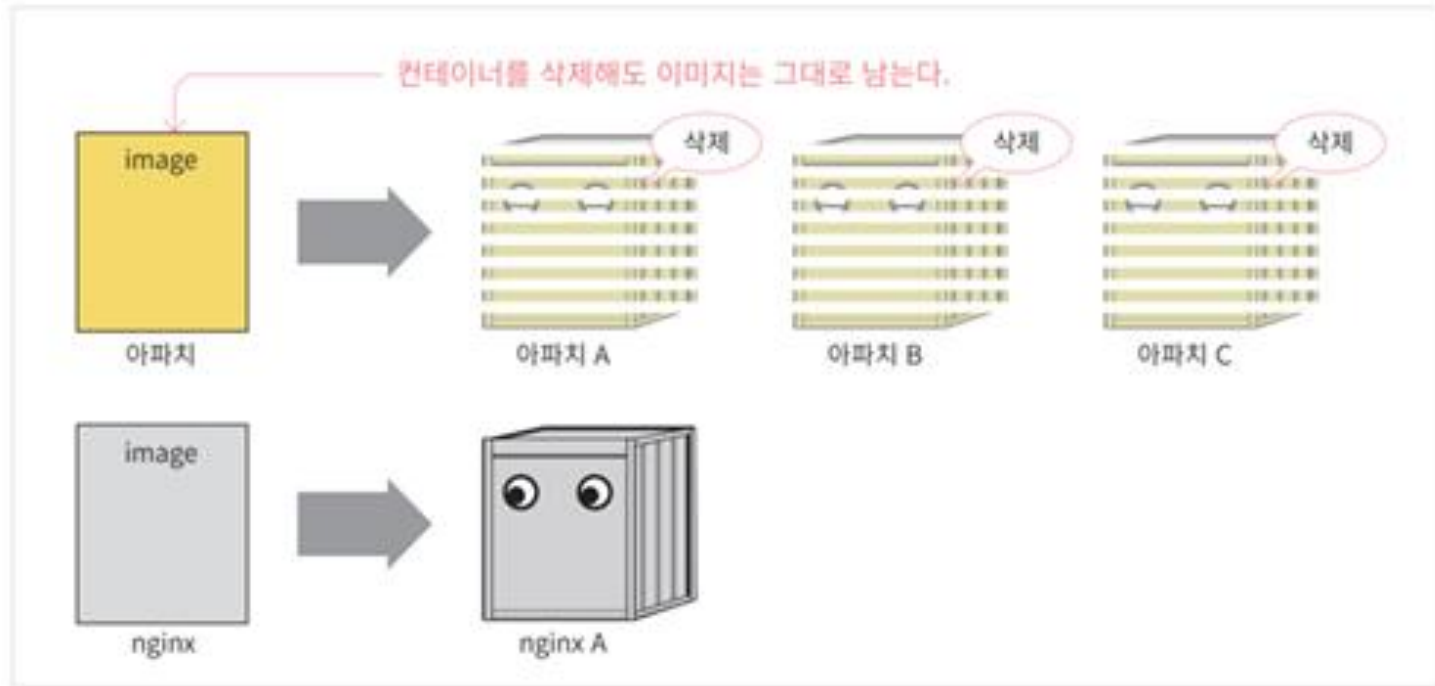
항목	내용
--name mysql000ex7	mysql000ex7이라는 이름으로 컨테이너를 생성
-dit	백그라운드에서 실행 및 키보드를 통해 컨테이너 내부의 파일 시스템을 조작
-e MYSQL_ROOT_PASSWORD=	MySQL의 루트 패스워드를 지정
mysql	MySQL 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

컨테이너 실행

6절 - 이미지 삭제

이미지 삭제

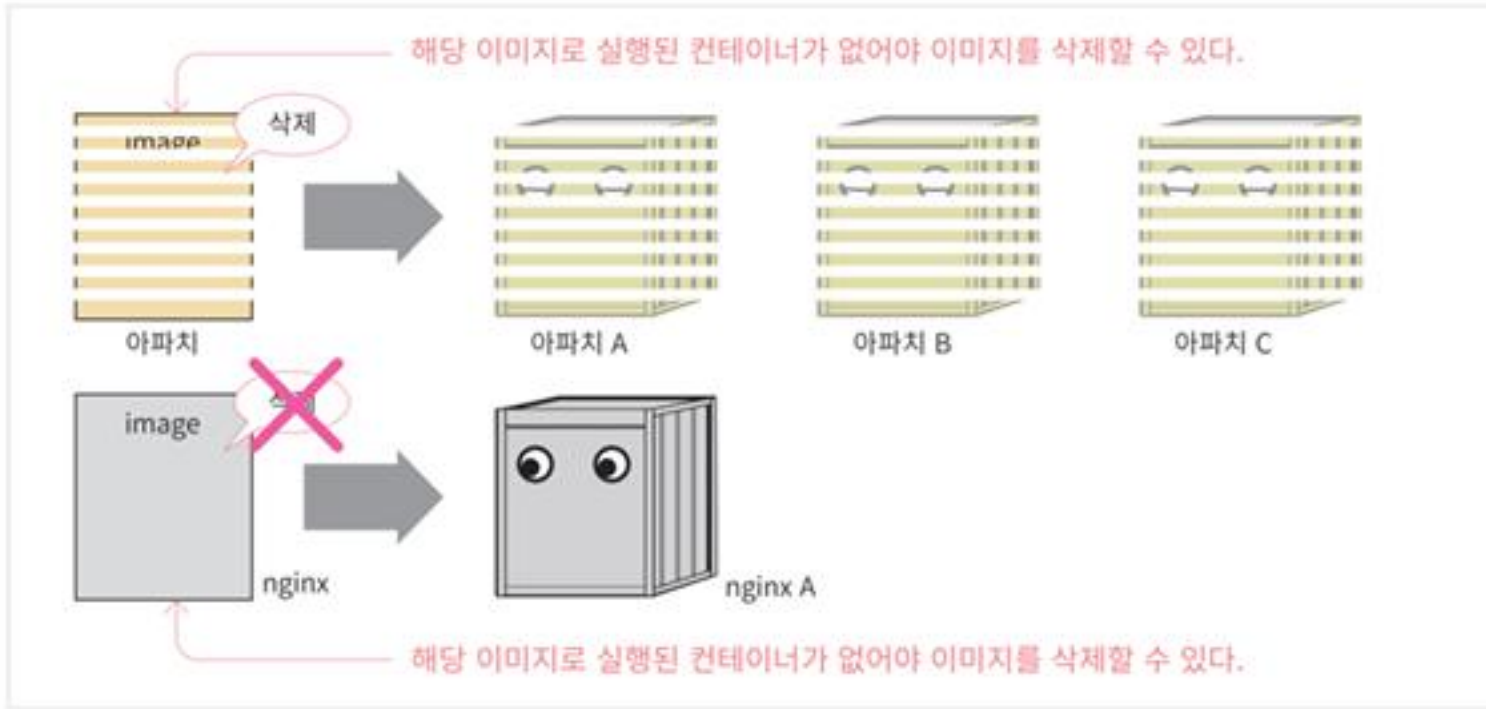
- 컨테이너를 삭제해도 이미지는 남아있으므로 불필요한 이미지는 즉시 삭제



- 컨테이너의 삭제와 이미지의 삭제

이미지 삭제

- 해당 이미지의 컨테이너가 남아 있으면 이미지 삭제 안됨



- 해당 이미지로 실행된 컨테이너가 있으면 이미지를 삭제할 수 없다.

이미지 삭제

- 이미지 삭제 명령

```
docker image rm 이미지명 [이미지명 . . .] 또는 이미지ID
```

- 이미지 조회 명령

```
docker image ls
```

이미지 목록의 주요 항목

항목	내용
REPOSITORY	이미지 이름
TAG	버전 정보. 이미지를 내려받을 때 따로 지정하지 않으면 latest(최신 버전)를 내려받는다.
IMAGE ID	이미지 식별자. 본래는 64글자이지만 앞에서부터 12글자만 출력한다. 이 12글자만으로도 식별자 역할을 수행할 수 있다.
CREATED	이미지 생성 후 경과된 시간
SIZE	이미지의 전체 용량

이미지 삭제

- <실습> 다운받은 모든 이미지 삭제하기

